

Web Application Development

Lecture 2: JSX & Rendering

SWE 381 – Web Application Development

Dr. Ohoud Alharbi Dr. Achraf Gazdar

Department of Software Engineering
College of Computer and Information Sciences
King Saud University

Term 2 – 2025/2026

Learning Objectives

By the end of this lecture, you will be able to:

- Compare JSX and HTML syntax and list key differences
- Use `className`, `htmlFor`, and **self-closing tags** correctly in JSX
- Embed JavaScript expressions in JSX using `{ }`
- Apply **ternary** (`? :`) and **&&** operators for conditional rendering
- Render lists from arrays using `array.map()`
- Explain **why keys are required** and choose the right key value
- Chain `filter()` + `map()` to display filtered data
- Handle **empty state** gracefully in UI components

Agenda

- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with `map()`
- 5 Keys – Why & How
- 6 `filter()` + `map()` Chaining
- 7 Empty State Handling
- 8 Practical Examples & Summary

Outline

- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with map()
- 5 Keys – Why & How
- 6 filter() + map() Chaining
- 7 Empty State Handling
- 8 Practical Examples & Summary

What is JSX? – A Quick Recap

W3Schools – JSX Intro

JSX stands for **JavaScript XML**. It allows us to write HTML directly within JavaScript. JSX is an extension of the JavaScript language based on ES6, and is translated into regular JavaScript at runtime by a compiler (Babel / Vite).

With JSX (W3Schools example):

```
1 const myElement = <h1>I Love JSX!</h1>;  
2  
3 createRoot(document.getElementById('root'))  
4   .render(myElement);  
5
```

Without JSX:

```
1 const myElement =  
2   React.createElement('h1', {}, 'I do not use JSX!');  
3  
4 createRoot(document.getElementById('root'))  
5   .render(myElement);
```



You are not required to use JSX...

... but JSX makes React code significantly more **readable** and is used in virtually every real-world React project. W3Schools: “JSX allows us to write HTML directly within the JavaScript code.”

JSX vs HTML – Comparison Table

Feature	HTML	JSX	Reason
CSS class	<code>class="card"</code>	<code>className="card"</code>	<code>class</code> is a JS reserved word
Label target	<code>for="email"</code>	<code>htmlFor="email"</code>	<code>for</code> is a JS reserved word
Self-close tags	<code>
</code> , <code></code>	<code>
</code> , <code></code>	JSX follows XML rules
Inline styles	<code>style="color:red"</code>	<code>style={{color:'red'}}</code>	JSX uses JS objects
Comments	<code><!-- comment --></code>	<code>{/* comment */}</code>	JSX uses JS syntax
Boolean attrs	<code>disabled</code>	<code>disabled={true}</code>	JSX is explicit
Event handlers	<code>onclick="fn()"</code>	<code>onClick={fn}</code>	camelCase in JSX
Root element	Not required	Required	JSX must return one element

JSX is NOT HTML!

JSX *looks* like HTML but compiles to JavaScript. Every “HTML-looking” difference in the table above is a real syntax rule that will cause errors if broken.

className – Replacing class

W3Schools – JSX Attributes

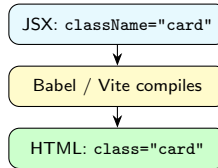
The class attribute is a much-used attribute in HTML, but since JSX is rendered as JavaScript, and the class keyword is a **reserved word** in JavaScript, you are not allowed to use it in JSX. Use attribute className instead.

HTML (wrong in JSX):

```
<!-- HTML -- works in browser -->
<div class="card highlight">
  <h2 class="title">Course</h2>
  <p class="body-text">SWE 381</p>
</div>
```

JSX (correct):

```
{/* JSX -- use className */}
function CourseCard() {
  return (
    <div className="card highlight">
      <h2 className="title">Course</h2>
      <p className="body-text">SWE 381</p>
    </div>
  );
}
```



Compiled automatically

When JSX is rendered, it **translates** className attributes into class attributes in the real DOM.

htmlFor and Self-Closing Tags

htmlFor – replacing for:

```
1 <!-- HTML -->
2 <label for="email">Email:</label>
3 <input type="email" id="email" />
```

```
1 { /* JSX -- use htmlFor */
2 function LoginForm() {
3   return (
4     <form>
5       <label htmlFor="email">
6         Email:
7       </label>
8       <input type="email" id="email" />
9     </form>
10   );
11 }
```

for is reserved in JS (for loops).

JSX uses **htmlFor** instead.

Self-closing tags:

```
1 <!-- HTML: optional slash -->
2 <br>
3 <hr>
4 
5 <input type="text">
```

```
1 { /* JSX: slash is REQUIRED */
2 function MyForm() {
3   return (
4     <>
5       <br />
6       <hr />
7       
8       <input type="text" />
9     </>
10   );
11 }
```



JSX follows XML rules

Every tag in JSX must be closed. JSX will throw an error if a tag is left open.

Inline Styles in JSX – Object Syntax

HTML inline style (string):

```
1 <!-- HTML: style is a string -->
2 <h1 style="color:blue; font-size:24px">
3   Hello
4 </h1>
```

JSX inline style (JS object):

```
1 /* JSX: style is an OBJECT */
2 /* Note the double {{ }} */
3 function Heading() {
4   return (
5     <h1 style={{
6       color: 'blue',
7       fontSize: '24px',           // camelCase!
8       backgroundColor: '#eee',   // camelCase!
9       marginTop: '10px'
10    }}>
11       Hello
12     </h1>
13   );
14 }
```

Or define styles as a variable:

```
1 const titleStyle = {
2   color: 'navy',
3   fontSize: '28px',
4   fontWeight: 'bold'
5 };
6
7 function PageTitle() {
8   return (
9     <h1 style={titleStyle}>
10       SWE 381
11     </h1>
12   );
13 }
```



CSS property names

CSS uses kebab-case: font-size, background-color.
JSX uses camelCase: fontSize, backgroundColor.

Outline

- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with `map()`
- 5 Keys – Why & How
- 6 `filter()` + `map()` Chaining
- 7 Empty State Handling
- 8 Practical Examples & Summary

Embedding Expressions – The { } Rule

W3Schools – JSX Expressions

With JSX you can write expressions inside curly braces { }. The expression can be a React variable, or property, or any other valid JavaScript expression. JSX will execute the expression and return the result.

```
1 function StudentCard() {  
2   const name    = "Ahmed Al-Farsi";  
3   const gpa     = 3.8;  
4   const year    = new Date().getFullYear();  
5   const isHonor = gpa >= 3.7;  
6  
7   return (  
8     <div className="card">  
9       /* Variable */  
10      <h2>{name}</h2>  
11  
12      /* Arithmetic expression */  
13      <p>Score: {gpa * 25}%</p>  
14  
15      /* Method call */  
16      <p>Year: {year}</p>  
17  
18      /* Ternary (covered next) */  
19      <p>Status: {  
20        isHonor ? "Honor Roll" : "Regular"  
21      }</p>  
22    </div>  
23  );  
24 }
```

Browser Output

Ahmed Al-Farsi
Score: 95%
Year: 2025
Status: Honor Roll

Valid inside { }:

- Variables: {name}
- Arithmetic: {gpa * 25}
- Calls: {getName()}
- Ternary: {x ? a : b}
- Template literal: {'Hi \${name}'}

NOT valid inside { }:

- if/else blocks
- for / while loops
- Variable declarations

Expressions in Attributes

W3Schools – JSX Attributes

You can also use JavaScript expressions as **attribute values**. Note that the attribute value is **not wrapped in quotes** when using expressions. If you use quotes, JSX will treat it as a string literal, not a JavaScript expression.

```
1 function UserAvatar() {
2   const user = {
3     name: "Sara Ahmed",
4     avatar: "sara.jpg",
5     isOnline: true,
6     size: 80
7   };
8
9   return (
10     <div>
11       /* Expression in src and alt */
12       <img
13         src={user.avatar}
14         alt={` ${user.name}'s avatar`}
15         width={user.size}
16         height={user.size}
17         className={user.isOnline ? "avatar-online" : "avatar-offline"}
18       />
19       /* String literals still use quotes */
20       <p className="username">{user.name}</p>
21       <span>{user.isOnline ? "Online" : "Offline"}</span>
22     </div>
23   );
24 }
```

Outline

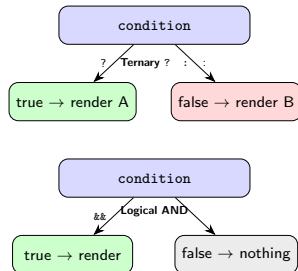
- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with `map()`
- 5 Keys – Why & How
- 6 `filter()` + `map()` Chaining
- 7 Empty State Handling
- 8 Practical Examples & Summary

Why No if/else Directly in JSX?

JSX is **syntactic sugar** for function calls. Since `if/else` is a **statement**, not an expression, it cannot live directly inside `{ }` in JSX markup.

Two solutions for inline conditionals:

- 1 **Ternary operator** `condition ? a : b`
Show *one of two* outcomes
- 2 **Logical `&&`** `condition && element`
Show *something or nothing*



Alternative: if/else before return

You can use `if/else` **before** the `return` statement to decide which JSX block to output:

```
1 function Greeting({ isLoggedIn }) {  
2   if (isLoggedIn) {  
3     return <h1>Welcome back!</h1>;  
4   }  
5   return <h1>Please sign in.</h1>;  
6 }
```

Ternary Operator ? : – W3Schools Example

W3Schools – React Conditional Rendering

Return the MadeGoal component if isGoal is true, otherwise return the MissedGoal component.

```
1 function MadeGoal() {
2   return <h1>Goal!</h1>;
3 }
4
5 function MissedGoal() {
6   return <h1>Missed!</h1>;
7 }
8
9 // Ternary picks ONE of two components
10 function Goal({ isGoal }) {
11   return (
12     <>
13       { isGoal
14         ? <MadeGoal />
15         : <MissedGoal />
16       }
17     </>
18   );
19 }
20
21 createRoot(document.getElementById('root'))
22   .render(<Goal isGoal={false} />);
```

isGoal={false}

Missed!

isGoal={true}

Goal!

Syntax recap:

```
condition ? <IfTrue /> : <IfFalse />
```

💡 Inline text too

```
{isLoggedIn ? 'Welcome!' : 'Sign in'}
```

Ternary works for text, JSX, or entire components.

The && Operator – W3Schools Example

W3Schools – React Conditional Rendering

We can use the && operator to conditionally render a React component. The right side of && will **only be rendered if the left side is true**.

```
1 function Garage({ cars }) {  
2   return (  
3     <>  
4       <h1>Garage</h1>  
5       {cars.length > 0 &&  
6         <h2>  
7           You have {cars.length} cars  
8           in your garage.  
9         </h2>  
10      }  
11    </>  
12  );  
13 }  
14  
15 // With cars  
16 const cars = ['Ford', 'BMW', 'Audi'];  
17 createRoot(...).render(  
18   <Garage cars={cars} />  
19 );  
20  
21 // Empty array -- h2 not shown  
22 const empty = [];  
23 createRoot(...).render(  
24   <Garage cars={empty} />  
25 );
```

🔗 cars = {'Ford','BMW','Audi'}

Garage

You have 3 cars in your garage.

🔗 cars = empty array

Garage

⚠️ Falsy number gotcha

If the left side is the number 0, React renders 0 instead of nothing. Use: `arr.length > 0 && ...` not `arr.length && ...`

Ternary vs && – When to Use Which

Use `?` : when you have TWO outcomes:

```
1 // Two different UI states
2 function AuthButton({ isLoggedIn }) {
3   return (
4     <button>
5       {isLoggedIn
6         ? 'Log Out'
7         : 'Log In'}
8     </button>
9   );
10 }
11
12 // Two different components
13 function Dashboard({ role }) {
14   return (
15     <>
16       {role === 'admin'
17         ? <AdminPanel />
18         : <UserPanel />}
19     </>
20   );
21 }
```

Use `&&` when you show or hide:

```
1 function Inbox({ messages }) {
2   return (
3     <div>
4       <h1>Inbox</h1>
5
6       {/* Show badge only if unread */}
7       {messages.length > 0 &&
8         <span className="badge">
9           {messages.length} unread
10         </span>
11       }
12
13       {/* Show warning only if empty */}
14       {messages.length === 0 &&
15         <p className="empty">
16           No messages.
17         </p>
18       }
19     </div>
20   );
21 }
```

Operator	Use when...	Alternatives
condition <code>?</code> A : B	Two different outcomes	if/else before return
condition <code>&&</code> A	Show or hide one thing	if before return

Outline

- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with `map()`**
- 5 Keys – Why & How
- 6 `filter()` + `map()` Chaining
- 7 Empty State Handling
- 8 Practical Examples & Summary

array.map() for Rendering Lists

W3Schools – React Lists

Let's create a simple list using the map() method. W3Schools example:

```
1 function MyCars() {  
2   const cars = ['Ford', 'BMW', 'Audi'];  
3  
4   return (  
5     <>  
6       <h1>My Cars:</h1>  
7       <ul>  
8         {cars.map((car) =>  
9           <li key={car}>I am a {car}</li>  
10        )}  
11      </ul>  
12    </>  
13  );  
14 }  
15  
16 createRoot(document.getElementById('root'))  
17   .render(<MyCars />);
```

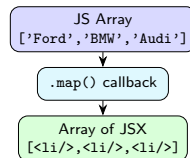
How map() works here:

- Takes each element from the array
- Returns a JSX element for each
- React renders the resulting array of JSX

Browser Output

My Cars:

- I am a Ford
- I am a BMW
- I am a Audi



Rendering an Array of Objects

```
1 // W3Schools array of objects example
2 const users = [
3   { id: 1, name: 'John', age: 30 },
4   { id: 2, name: 'Jane', age: 25 },
5   { id: 3, name: 'Ahmed', age: 22 },
6 ];
7
8 function UserList() {
9   return (
10     <ul>
11       {users.map(user =>
12         <li key={user.id}>
13           {user.name} is {user.age} years old
14         </li>
15       )}
16     </ul>
17   );
18 }
19
20 createRoot(document.getElementById('root'))
21   .render(<UserList />);
```

Note: each item maps to its own `<li>` element. The key prop (`user.id`) uniquely identifies each element (covered next).



Browser Output

- John is 30 years old
- Jane is 25 years old
- Ahmed is 22 years old

Component version – passing array as prop:

```
1 function App() {
2   return <UserList users={users} />;
3 }
4
5 function UserList({ users }) {
6   return (
7     <ul>
8       {users.map(user =>
9         <li key={user.id}>
10           {user.name} -- {user.age}
11         </li>
12       )}
13     </ul>
14   );
15 }
```

Outline

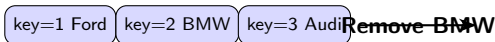
- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with `map()`
- 5 Keys – Why & How**
- 6 `filter()` + `map()` Chaining
- 7 Empty State Handling
- 8 Practical Examples & Summary

Why Do Keys Matter?

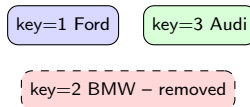
W3Schools – React Lists

Keys allow React to keep track of elements. This way, if an item is **updated** or **removed**, only that item will be re-rendered instead of the entire list.

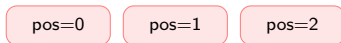
Before



After



Without keys – React re-renders ALL



Positions shifted! All items re-rendered.

With keys – React updates ONLY changed



Only key=2 removed. Others untouched.

Choosing the Right Key

W3Schools – React Lists

Generally, the key should be a **unique ID** assigned to each item. As a last resort, you can use the **array index** as a key.

Best: use a stable unique ID:

```
const courses = [
  { id: 'SWE381', title: 'Web App Dev' },
  { id: 'SWE401', title: 'SW Design' },
  { id: 'CS101', title: 'Intro to CS' },
];

// key = unique course code (stable!)
courses.map(c =>
  <li key={c.id}>{c.title}</li>
)
```

Last resort: use array index:

```
const fruits = ['apple', 'banana', 'cherry'];

// key = index (only if list never reorders)
fruits.map((fruit, index) =>
  <li key={index}>{fruit}</li>
)
```

Key rules summary:

- ✓ Must be **unique among siblings**
- ✓ Should be **stable** across renders
- ✓ Best: use **data IDs** from your database
- ✗ Don't use **random numbers** (`Math.random()`)
- ✗ Avoid index if list can **re-order** or items can be added/deleted

⚠ key is NOT a prop

key is a **special attribute** – it is never passed to the component as a prop. If you need the same value inside the component, pass it as a separate prop.

Without Key vs With Key – Console Warning

Missing key – causes warning:

```
function MyCars() {  
  const cars = ['Ford', 'BMW', 'Audi'];  
  return (  
    <ul>  
      {cars.map((car) =>  
        <li>{car}</li>  {/* NO KEY! */}  
      )}  
    </ul>  
  );  
}
```

Console warning

Warning: Each child in a list should have a unique "key" prop.

App still renders but React cannot optimise updates – it will re-render the entire list on every change.

With key – no warning, optimal:

```
function MyCars() {  
  const cars = ['Ford', 'BMW', 'Audi'];  
  return (  
    <ul>  
      {cars.map((car) =>  
        <li key={car}>{car}</li>  
      )}  
    </ul>  
  );  
}
```

No console warning

App renders correctly and React can efficiently update only the changed item.

W3Schools note: When using `map()` in React to create list items, each item **needs** a unique key prop.

Outline

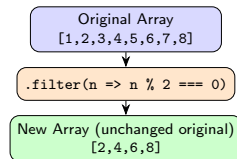
- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with `map()`
- 5 Keys – Why & How
- 6 `filter()` + `map()` Chaining**
- 7 Empty State Handling
- 8 Practical Examples & Summary

filter() – Selecting Array Elements

W3Schools – ES6 Array Methods

The `filter()` method creates a **new array** filled with elements that pass a test (provided as a function). It does not change the original array.

```
1 const numbers = [1, 2, 3, 4, 5, 6, 7, 8];
2
3 // filter returns only elements where
4 // the callback returns true
5 const evens = numbers.filter(
6   n => n % 2 === 0
7 );
8 console.log(evens); // [2, 4, 6, 8]
9
10 // Filter an array of objects
11 const students = [
12   { id: 1, name: 'Ahmed', gpa: 3.8 },
13   { id: 2, name: 'Sara', gpa: 2.9 },
14   { id: 3, name: 'Khalid', gpa: 3.5 },
15   { id: 4, name: 'Noura', gpa: 2.4 },
16 ];
17
18 // Only students with gpa >= 3.0
19 const passing = students.filter(
20   s => s.gpa >= 3.0
21 );
22 // [Ahmed (3.8), Khalid (3.5)]
```



Immutable!

`filter()` always returns a **new array**. The original is never modified – key for React's immutability pattern.

Chaining filter() + map()

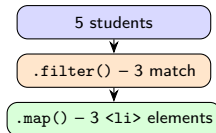
```
const students = [
  { id: 1, name: 'Ahmed', gpa: 3.8, dept: 'SWE' },
  { id: 2, name: 'Sara', gpa: 2.9, dept: 'CS' },
  { id: 3, name: 'Khalid', gpa: 3.5, dept: 'SWE' },
  { id: 4, name: 'Noura', gpa: 2.4, dept: 'IS' },
  { id: 5, name: 'Ali', gpa: 3.2, dept: 'SWE' },
];

function SWEStudentList() {
  return (
    <div>
      <h2>SWE Students (GPA >= 3.0)</h2>
      <ul>
        {students
          .filter(s =>
            s.dept === 'SWE' && s.gpa >= 3.0
          )
          .map(s =>
            <li key={s.id}>
              {s.name} -- GPA: {s.gpa}
            </li>
          )
        }
      </ul>
    </div>
  );
}
```

Browser Output

SWE Students (GPA $\geq$ 3.0)

- Ahmed – GPA: 3.8
- Khalid – GPA: 3.5
- Ali – GPA: 3.2



Filter and Map – Separate Variables for Clarity

Inline chain (concise):

```
function HonorList({ students }) {  
  return (  
    <ul>  
      {students  
        .filter(s => s.gpa >= 3.7)  
        .map(s =>  
          <li key={s.id}>  
            {s.name} -- {s.gpa}  
          </li>  
        )  
      }  
    </ul>  
  );  
}
```

Separate variables (readable):

```
1 function HonorList({ students }) {  
2   // Step 1: filter  
3   const honorStudents = students  
4     .filter(s => s.gpa >= 3.7);  
5  
6   // Step 2: map to JSX  
7   const listItems = honorStudents  
8     .map(s =>  
9     <li key={s.id}>  
10       {s.name} -- {s.gpa}  
11     </li>  
12   );  
13  
14   // Step 3: render  
15   return <ul>{listItems}</ul>;  
16 }
```



Which style to use?

Both produce **identical results**. Use the **inline chain** for short, simple operations. Use **separate variables** when the logic is complex or when you need to reuse the filtered data in multiple places (e.g. showing a count).

Outline

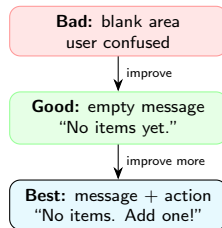
- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with map()
- 5 Keys – Why & How
- 6 filter() + map() Chaining
- 7 Empty State Handling**
- 8 Practical Examples & Summary

What is Empty State?

Empty state occurs when an array or data source has **no items to display**. Without handling it, the user sees a blank area with no explanation.

Common empty state scenarios:

- A to-do list with no tasks yet
- Search results with zero matches
- A cart with no items
- A notification list when all are read
- Data still loading from the server



Good UX principle

Always tell users **why** the list is empty and, where possible, **what they can do** about it. A blank page is never acceptable.

Empty State with && and Ternary

Using && (W3Schools pattern):

```
1 function Garage({ cars }) {  
2   return (  
3     <>  
4     <h1>Garage</h1>  
5  
6     /* Show list only if not empty */  
7     {cars.length > 0 &&  
8       <ul>  
9         {cars.map(c =>  
10          <li key={c}>{c}</li>  
11          )}  
12        </ul>  
13      }  
14  
15     /* Show message only if empty */  
16     {cars.length === 0 &&  
17       <p>Your garage is empty.</p>  
18     }  
19   </>  
20 );  
21 }
```

Using ternary (cleaner):

```
1 function Garage({ cars }) {  
2   return (  
3     <>  
4     <h1>Garage</h1>  
5     {cars.length > 0  
6       ? (  
7         <ul>  
8           {cars.map(c =>  
9             <li key={c}>{c}</li>  
10            )}  
11         </ul>  
12       )  
13     : (  
14       <p className="empty-msg">  
15         No cars yet. Add one!  
16       </p>  
17     )  
18   }  
19 </>  
20 );  
21 }
```

Empty State – Full Practical Example

```
1 const allCourses = [  
2   { id: 1, title: 'Web App Dev',   dept: 'SWE', credits: 3 },  
3   { id: 2, title: 'AI Basics',    dept: 'CS',  credits: 2 },  
4   { id: 3, title: 'SW Design',     dept: 'SWE', credits: 3 },  
5 ];  
6  
7 function CourseList({ dept }) {  
8   const filtered = allCourses.filter(c => c.dept === dept);  
9  
10  return (  
11    <div className="course-list">  
12      <h2>Courses -- Dept: {dept}</h2>  
13  
14      {filtered.length > 0  
15        ? (  
16          <ul>  
17            {filtered.map(c => (  
18              <li key={c.id}>  
19                <strong>{c.title}</strong> ({c.credits} cr)  
20                </li>  
21              )})  
22            </ul>  
23          )  
24          : (  
25            <p className="empty">  
26              No courses found for department <em>{dept}</em>.  
27            </p>  
28          )  
29        }  
30  
31        <p>Showing {filtered.length} of {allCourses.length} courses.</p>  
32      </div>  
33    );  
34  }
```

Empty State – Output Comparison

🔍 dept = "SWE"

Courses – Dept: SWE

- Web App Dev (3 cr)
- SW Design (3 cr)

Showing 2 of 3 courses.

🔍 dept = "IS" (no match)

Courses – Dept: IS

No courses found for department IS.

Showing 0 of 3 courses.

What makes a good empty state:

- Show a **clear message** – not just blank
- Mention **what was searched/filtered**
- Show **count context** – “0 of 3 courses”
- Optionally offer a **call to action**

💡 Also handle loading state

When fetching from an API:

```
1 {isLoading
2   ? <p>Loading...</p>
3   : data.length === 0
4     ? <p>No results.</p>
5     : data.map(...)
6 }
```

Outline

- 1 JSX vs HTML – Syntax Differences
- 2 Expressions in { }
- 3 Ternary & && Conditional Rendering
- 4 Rendering Lists with `map()`
- 5 Keys – Why & How
- 6 `filter()` + `map()` Chaining
- 7 Empty State Handling
- 8 Practical Examples & Summary

Putting It All Together – Student Dashboard

```
1 const students = [  
2   { id: 1, name: 'Ahmed', gpa: 3.8, active: true },  
3   { id: 2, name: 'Sara', gpa: 2.9, active: false },  
4   { id: 3, name: 'Khalid', gpa: 3.5, active: true },  
5   { id: 4, name: 'Noura', gpa: 2.4, active: true },  
6 ];  
7  
8 function StudentDashboard({ minGpa }) {  
9   // 1. filter -- only active students above minGpa  
10  const qualified = students  
11    .filter(s => s.active && s.gpa >= minGpa);  
12  
13  return (  
14    <div className="dashboard">  
15      <h1>Student Dashboard</h1>  
16      <p>Filter: GPA >= {minGpa}</p>  
17  
18      /* 2. empty state handling */  
19      {qualified.length === 0  
20        ? <p className="empty">No students match this filter.</p>  
21        : <ul>  
22          /* 3. map with key -- uses id */  
23          {qualified.map(s => (  
24            <li key={s.id}>  
25              className={s.gpa >= 3.7 ? "honor" : "regular">  
26                {s.name} -- GPA: {s.gpa}  
27                /* 4. ternary + ES inline */  
28                {s.gpa >= 3.7 && <span>      Honor Roll</span>  
29              </li>  
30            )}}  
31          </ul>  
32        )  
33      </div>  
34    );  
35  }
```

Component usage:

```
// minGpa = 3.0 (Ahmed, Khalid match)
<StudentDashboard minGpa={3.0} />

// minGpa = 4.0 (no one matches)
<StudentDashboard minGpa={4.0} />
```

Concepts used in this slide:

- 1 filter() – selects active students above minGpa
- 2 Empty state ternary – handles zero results
- 3 map() with key – renders the list
- 4 className (JSX attribute) – applies styling
- 5 Ternary in className – dynamic class
- 6 && inline – shows honor badge conditionally

 minGpa={3.0}

Student Dashboard

Filter: GPA $\geq$ 3.0

- Ahmed – GPA: 3.8 ★ **Honor Roll**
- Khalid – GPA: 3.5

 minGpa={4.0}

Student Dashboard

Filter: GPA $\geq$ 4.0

No students match this filter.

Lecture 2 – Key Takeaways

- 1 **JSX $\neq$ HTML**: use `className`, `htmlFor`, self-closing tags, and JS object styles
- 2 **Expressions in `{ }`**: any valid JS expression – variables, arithmetic, calls, ternary
- 3 Attribute values use `{expr}` (not quotes) for dynamic content
- 4 **Ternary `? : &&`**: renders one of two outcomes; **`&&`** shows or hides one element
- 5 `array.map()` transforms each data item into a JSX element for rendering
- 6 **Keys** (`key={item.id}`) let React track list items and re-render only what changed
- 7 Always prefer **stable unique IDs** as keys; use array index only as a last resort
- 8 **Chain** `filter()` $\rightarrow$ `map()` to render a filtered subset of data
- 9 Always handle **empty state**: show a helpful message when the list is empty

Next Lecture: React Components – Props, Component Hierarchy & Data Flow

Practice Exercises

- 1 Create a JSX component `ProfileCard` with an image (`<img />`), a heading, and a paragraph. Apply CSS classes using `className` and link a label to an input using `htmlFor`.
- 2 Build a `LoginStatus` component that receives an `isLoggedIn` boolean prop and renders "Welcome back!" or "Please sign in." using the ternary operator.
- 3 Create a `NotificationBadge` component that shows a red badge with the count **only when** `count > 0`, using the `&&` operator.
- 4 Given an array of 5 course objects (`id`, `title`, `credits`), render them as an unordered list using `map()`. Add a key prop and confirm no console warning appears.
- 5 Extend the course list: use `filter()` to display only courses with `credits >= 3`, then `map()` the result. Show the total filtered count.
- 6 Add **empty state handling** to the course list: if no courses match the filter, show the message "No courses found." Test with a filter that matches nothing.
- 7 **Challenge:** Build a `StudentTable` component that takes an array of student objects. Render a `<table>` using `map()`, show only students with `gpa >= 3.0` using `filter()`, mark honor students (`gpa >= 3.7`) with a star using `&&`, and show "No honor students." if the filtered list is empty.